

For a **2 years experienced Java Developer**, interviewers usually expect:

- Strong understanding of **Core Java fundamentals**
- Ability to explain concepts with **real project examples**
- Knowledge of **Collections, Streams, Multithreading, JVM, GC**
- Problem-solving using Java
- Some internal working knowledge

Here's a structured list of the **most asked Core Java interview questions**.

Java Fundamentals

OOPs Concepts

- What are the 4 pillars of OOP?
 - Difference between abstraction and encapsulation
 - Difference between interface and abstract class
 - Can interface have methods with body?
 - What is polymorphism?
 - Compile-time vs runtime polymorphism
 - Method overloading vs overriding
 - Why multiple inheritance is not supported in Java classes?
 - What is association, aggregation, composition?
-

Class, Object, Memory

- What happens when an object is created?
- What is stored in Heap and Stack?
- Difference between stack memory and heap memory
- What is constructor?
- Constructor chaining
- Can constructor be private?
- What is `this` keyword?
- What is `super` keyword?
- Static variable vs instance variable
- Static block vs instance block

- Why main method is static?
-

String Questions (Very Important)

Basic

- Why String is immutable?
- Difference between `String`, `StringBuilder`, `StringBuffer`
- What is String Constant Pool?
- Difference between:

```
String s1 = "abc";  
String s2 = new String("abc");
```

Internal Working

- How `equals()` works in String?
- Why String is final?
- What is interning?
- Difference between `==` and `.equals()`

Coding

- Reverse string
 - Check palindrome
 - Count characters
 - Find duplicates
 - Anagram check
-

Collections Framework (MOST ASKED)

List

- Difference between `ArrayList` and `LinkedList`
- Internal working of `ArrayList`
- Why `ArrayList` is faster for read?
- How `LinkedList` works internally?

Set

- Difference between `HashSet`, `LinkedHashSet`, `TreeSet`
- How `HashSet` avoids duplicates?
- What is hashing?

Map

- Difference between `HashMap`, `Hashtable`, `ConcurrentHashMap`
- Internal working of `HashMap`
- What happens during hash collision?
- What is bucket?
- What is load factor?
- What is rehashing?
- Why Java 8 uses balanced tree in `HashMap`?
- Why `HashMap` allows one null key?

Comparable & Comparator

- Difference between `Comparable` and `Comparator`
- How sorting works internally?

equals() and hashCode() (VERY IMPORTANT)

- Why override `hashCode` when `equals` is overridden?
- Contract between `equals` and `hashCode`
- What happens if `hashCode` is not overridden?
- Can two objects have same `hashCode`?
- Can `hashCode` be different for equal objects?

Example expected:

HashSet<Employee>

Why duplicate objects still inserted?

Exception Handling

- Difference between checked and unchecked exception
 - Difference between throw and throws
 - Final vs finally vs finalize
 - Can finally block skip execution?
 - What happens if exception occurs in finally?
 - Try-with-resources
 - Custom exception
-

Multithreading (HIGHLY ASKED)

Basics

- Process vs Thread
- Ways to create thread
- Runnable vs Callable
- Thread lifecycle

Synchronization

- What is synchronization?
- What is race condition?
- What is deadlock?
- How to avoid deadlock?
- synchronized method vs synchronized block

Advanced

- What is volatile?

- What is atomicity?
 - What is ExecutorService?
 - Difference between wait(), sleep(), notify()
 - Difference between synchronized and Lock
 - What is ThreadLocal?
 - What is ConcurrentHashMap?
-

Java 8 Features (VERY IMPORTANT)

Lambda

- What is functional interface?
- What is lambda expression?
- Why lambda introduced?

Streams

- Difference between Collection and Stream
- Intermediate vs terminal operation
- Difference between map and flatMap
- filter vs map
- sorted vs distinct
- findFirst vs findAny
- anyMatch/allMatch/noneMatch
- Why max/min return Optional?
- Difference between Optional.of and Optional.ofNullable

Coding Questions

- Find duplicates using stream
- Count frequency
- Find second highest salary
- Group by department
- Find highest salary employee
- Convert list to map
- Remove duplicates

Example:

```
employees.stream()  
    .collect(Collectors.groupingBy(Employee::getDept));
```

JVM Internals (VERY IMPORTANT FOR 2 YEARS)

JVM Architecture

- What are JVM, JRE, JDK?
- ClassLoader types
- What is bytecode?
- What happens when Java program runs?

Memory Areas

- Heap
- Stack
- Metaspace
- Method Area
- PC Register

Garbage Collection

- What is Garbage Collection?
 - Minor GC vs Major GC vs Full GC
 - Young Generation vs Old Generation
 - What is memory leak?
 - Can Java have memory leak?
 - What objects are eligible for GC?
 - What is stop-the-world event?
-

Immutable Class

- How to create immutable class?
- Why immutable objects are thread-safe?
- Why String immutable?

Expected code:

```
final class Employee {  
    private final String name;  
}
```

Serialization

- What is serialization?
 - What is `serialVersionUID`?
 - transient keyword
 - Externalization vs Serialization
-

Keywords

Frequently Asked

- final
 - finally
 - finalize
 - static
 - volatile
 - transient
 - synchronized
-

Important Internal Working Questions

These are frequently asked for experienced developers:

HashMap Internal

Explain:

1. hashCode generation
2. bucket calculation
3. collision handling
4. linked list
5. treeification in Java 8

ConcurrentHashMap

- Segment locking (Java 7)
- CAS + bucket locking (Java 8)

ArrayList Internal

- Dynamic resizing
 - Capacity increase
-

Coding Questions Asked Frequently

String

- Reverse string
- Reverse words
- Count vowels
- Palindrome

Array

- Find duplicates
- Second largest
- Missing number
- Rotate array

Stream-based

- Frequency map
 - Grouping
 - Sorting custom object
 - Find nth highest salary
-

Scenario-Based Questions

These are common for 2 years experience:

- Where did you use multithreading?
 - Why did you choose ConcurrentHashMap?
 - Have you faced memory issue?
 - How did you optimize performance?
 - Why use StringBuilder?
 - Difference between REST call sync vs async?
 - How did you handle exception globally?
-

Questions They Ask to Judge Experience Level

Beginner-Level

- What is class?
- What is inheritance?

2-Year Expected Level

- Explain HashMap internals
- Explain JVM memory
- Explain synchronization issue you faced
- Explain stream optimization
- Explain GC issue/debugging

- Explain immutable object benefits
-

Most Important Topics Priority Order

Focus heavily on these:

1. Collections
 2. HashMap internals
 3. Streams API
 4. Multithreading
 5. JVM + Garbage Collection
 6. Exception handling
 7. String internals
 8. equals/hashCode
 9. Java 8
 10. Concurrent collections
-

Most Frequently Asked One-Line Questions

- Why String immutable?
 - Why HashMap not thread-safe?
 - Why wait() inside synchronized?
 - Why Optional introduced?
 - Why ConcurrentHashMap faster?
 - Why fail-fast iterator?
 - Why Java pass-by-value?
 - Why multiple inheritance not supported?
 - Why equals and hashCode both needed?
 - Why stream cannot be reused?
-

Must Prepare Practical Explanations

For every topic prepare:

- Definition
- Internal working

- Real project usage
- Advantages/disadvantages
- One example

Example:

"We used ConcurrentHashMap in our crude oil tracking system because multiple users updated shipment data simultaneously."

That kind of answer makes you sound experienced instead of theoretical.